

Interference-Aware Routing Schemes

Autonomous Networks Research Group (ANRG)
University of Southern California

Contents

1	Overview	3
2	Testing Environment	3
2.1	Network Topologies	3
2.1.1	Grid Networks	3
2.1.2	Benchmark Scenarios	4
2.2	DAG Structures	4
2.2.1	Fork-Join (Small DAG, 8 tasks)	4
2.2.2	Multi-Stage Pipeline (Large DAG, 30 tasks on 4×4 ; 60 tasks on 7×7)	4
2.2.3	Win-Regime Test DAGs	5
2.3	What a Schedule Looks Like	5
2.3.1	HEFT Placement	5
2.3.2	Example: demo_simple Trace	6
2.3.3	Example: Parallel Corridors Trace (GSD)	6
2.3.4	Simulation Fidelity	6
2.4	Evaluation Seeds and Averaging	7
2.5	Workload Heterogeneity	7
2.5.1	Heterogeneous Data Sizes (2–30 MB)	7
2.5.2	Heterogeneous Compute Costs (150–1000 compute units)	7
2.6	HEFT Scheduler Variants	8
2.6.1	HEFT (Calibrated) — Runtime-Routing BW for All Pairs	8
2.6.2	HEFT-1 — Penalty Rate for Non-Adjacent Pairs	9
2.6.3	HEFT-2 — Widest-Path Bottleneck Rate for All Pairs	9
2.6.4	Comparison	9
3	Baseline Schemes	10
3.1	D — Direct Link	10
3.2	W — Widest-Path	10
3.3	S — Shortest-Path (Minimum Transfer Delay)	11
3.4	SH — Shortest-Hop (Minimum Hop Count)	11
4	Static Greedy Schemes (GS, GC, GB, GO)	12
4.1	Scoring Function (All Four Variants)	12
4.2	Concurrency Detection	13
4.3	GS — Greedy by Start Time	13
4.4	GC — Greedy by Criticality	13
4.5	GB — Greedy by Bytes	13

4.6	GO — Greedy by Overlap	14
5	Dynamic Greedy Scheme	14
5.1	GSD — Greedy by Start Time, Dynamic	14
6	Deferral Scheme	15
6.1	GSD-D — Dynamic + Deferral	15
6.2	Deferral Mechanism	15
6.3	Parameters	16
6.4	When Deferral Helps	16
7	Configuration Reference	16
7.1	CLI	16
7.2	YAML	16
7.3	Shared Parameters	17
8	Summary	17

1 Overview

When tasks in a DAG are placed on different nodes, data transfers traverse multi-hop wireless paths where concurrent transmissions interfere with each other. The **routing scheme** decides *which path* each transfer takes and—for deferral schemes—*when* it begins.

The schemes fall into four families:

Family	Schemes	When routes are decided
Baseline	D, W, S, SH	Once, at topology load time (cached)
Static greedy	GS, GC, GB, GO	Once, at DAG injection (after HEFT)
Dynamic greedy	GSD	At each <code>transfer_start</code> event
Deferral	GSD-D	At <code>transfer_start</code> , may postpone

2 Testing Environment

This section describes the network topologies, DAG structures, schedules, and simulation parameters used throughout the ncsim evaluation suite. Understanding the test environment is essential for interpreting when and why different routing schemes win.

2.1 Network Topologies

2.1.1 Grid Networks

The primary evaluation topologies are regular 2D grids with optional diagonal links. Nodes are placed at uniform spacing (40 m in the evaluation suite) with bidirectional links between neighbors.

Topology	Nodes	Links	Notes
4×4 grid	16	68	Grid + checkerboard diagonals, heterogeneous compute capacities
7×7 grid	49	240	Grid + checkerboard diagonals, heterogeneous compute capacities
Parallel corridors	4	8	Two equal-length 2-hop paths (<code>n_src</code> $\rightarrow$ <code>n_top/bot</code> $\rightarrow$ <code>n_dst</code>)
Spine + bypass	4	8	Dense spine corridor plus separated bypass path
Dense 3×3	9	24	15 m spacing; adjacent links interfere at proximity radius=20

RF parameters (csma_bianchi mode): 5 GHz band, 20 MHz channel, $P_{tx} = 20$ dBm, path-loss exponent $n = 3.0$. At 40 m spacing, adjacent-link PHY rate is ≈ 8.6 MB/s; diagonal links (≈ 56.6 m) have lower PHY rates.

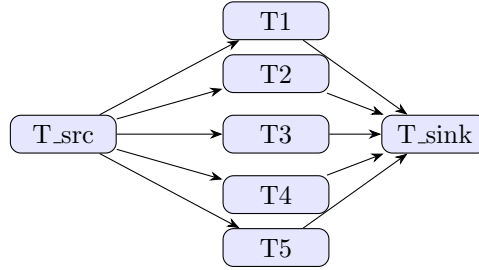
Scenario	Description
demo_simple.yaml	2 nodes, 1 link; T0 (cc=100) $\rightarrow$ T1 (cc=200) via 50 MB transfer.
bandwidth_contention.yaml	Two transfers on the same link; validates BW fair-sharing.
interference_aware_routing.yaml	X-topology: two crossing flows, each with spine vs. corridor path choices.
parallel_spread.yaml	Fork DAG on a grid; multiple simultaneous transfers.

2.1.2 Benchmark Scenarios

2.2 DAG Structures

The simulator executes DAGs (Directed Acyclic Graphs) where nodes are compute tasks and edges carry data between them. Three DAG shapes are used across evaluations:

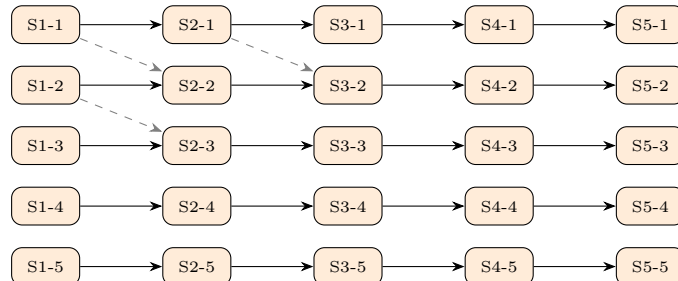
2.2.1 Fork-Join (Small DAG, 8 tasks)



- 1 source task fans out to 6 parallel workers, which all converge on 1 sink.
- **12 edges** total (6 fan-out + 6 fan-in).
- **Transfer sizes:** 2–30 MB (heterogeneous), drawn from a fixed distribution per seed.
- **Compute costs:** 150–1000 compute units (heterogeneous).
- All 6 fan-out transfers fire *simultaneously* when T_src completes.
- The 6 fan-in transfers fire in a scattered pattern as each worker finishes.

This DAG is designed to create **burst contention**: the fan-out phase generates 6 simultaneous transfers that compete for paths.

2.2.2 Multi-Stage Pipeline (Large DAG, 30 tasks on 4×4 ; 60 tasks on 7×7)



- 5–6 stages, each with multiple tasks (6 tasks/stage on 4×4 , 10 tasks/stage on 7×7).
- **48 edges** (4×4), **102 edges** (7×7).
- Stage-to-stage edges run in parallel; cross-connections (dashed) add inter-row dependencies that stagger transfer start times.
- **Transfer sizes:** 2–30 MB; **Compute costs:** 150–1000 units.
- Each stage transition fires 5–10 simultaneous transfers, sustaining high network utilization throughout execution.

2.2.3 Win-Regime Test DAGs

These are hand-crafted DAGs used in `tests/test_win_regimes.py` to isolate specific advantages of interference-aware routing.

Test DAG	Structure	Purpose
2-flow fork	$T_0 \text{ (n_src)} \rightarrow T_1, T_2 \text{ (n_dst)}$	2 simultaneous transfers on same (src, dst) pair
4-flow fork	$T_0 \text{ (n_src)} \rightarrow T_1\text{--}T_4 \text{ (n_dst)}$	4 simultaneous transfers; larger contention
Wide fork (mixed)	$T_src \rightarrow 5 \text{ sinks at different nodes}$	Heterogeneous data sizes (5–200 MB)
Critical-path	$T_src \rightarrow \{T_critical \text{ (cc=5000)}, T_noncrit \text{ (cc=10)}\}$	One edge on critical path, one off
Pipeline (3×3)	$3 \text{ stages} \times 3 \text{ nodes} + \text{cross-edges}$	Dense concurrent transfers for deferral testing

2.3 What a Schedule Looks Like

After the routing scheme and HEFT scheduler run, each execution produces a *schedule*: an assignment of tasks to nodes with a timing trace.

2.3.1 HEFT Placement

HEFT (Heterogeneous Earliest Finish Time) assigns each task to the node where it finishes earliest, accounting for:

- **Compute time:** $\text{task_runtime} = \text{compute_cost} / \text{node.compute_capacity}$
- **Transfer time (estimated):** $\text{transfer_time} = \text{data_size} / \text{path_bandwidth} + \text{path_latency}$
- **Node queue:** tasks are queued FIFO; HEFT accounts for when the node finishes its current task

HEFT uses the routing model’s `get_path_bandwidth()` to estimate transfer times during placement. All routing schemes delegate this to `WidestPathRouting` (max bottleneck bandwidth), so placement decisions are independent of the chosen routing scheme.

2.3.2 Example: demo_simple Trace

For the 2-node scenario with T0 (cc=100, on n0, capacity=100) → T1 (cc=200, on n0, capacity=100) with a 50 MB transfer on l01 (BW=100, lat=0.001):

```
HEFT assigns both tasks to n0 (same-node transfer, no network needed).
t=0.000 task_start      T0  on n0
t=1.000 task_complete   T0  on n0    (100 / 100 = 1.0 s)
t=1.000 transfer_start  T0->T1  (same node: immediate)
t=1.000 task_start      T1  on n0
t=3.000 task_complete   T1  on n0    (200 / 100 = 2.0 s)
Makespan: 3.000 s
```

2.3.3 Example: Parallel Corridors Trace (GSD)

Fork DAG: T0 (cc=1000) on n_src (cap=200) fans out to T1, T2 on n_dst. Two corridors (top: n_src → n_top → n_dst; bot: similar), each link BW=100 MB/s.

```
t=0.000 task_start      T0  on n_src
t=5.000 task_complete   T0  on n_src  (1000/200 = 5.0 s)
t=5.000 transfer_start  T0->T1  via bot corridor (GSD: no active links ->
bot)
t=5.000 transfer_start  T0->T2  via top corridor (GSD: bot active -> pick
top)
-- each transfer: 100 MB / 100 MB/s = 1.0 s + 0.002 latency --
t=6.002 transfer_complete T0->T1 and T0->T2
t=6.002 task_start      T1  on n_dst
t=6.007 task_complete   T1
t=6.007 task_start      T2  on n_dst (FIFO queue)
t=6.012 task_complete   T2
Makespan: 6.012 s

SP (both flows on same corridor):
effective BW = 100/2 = 50 MB/s per flow -> transfer = 2.0 s
Makespan: 7.012 s [GSD saves 1.0 s = 14%]
```

2.3.4 Simulation Fidelity

The ncsim discrete-event simulator captures:

- **Node queuing:** FIFO single-server queue per node; tasks wait if a node is busy. This is why T2 starts 0.005s after T1 in the example above—both are assigned to n_dst.
- **Bandwidth sharing:** N concurrent flows on a link each get $BW \times \text{interference_factor}/N$. Recalculated at every transfer start/complete event.
- **Multi-hop paths:** bottleneck bandwidth = minimum BW along path; total latency = sum of per-link latencies.
- **Reproducibility:** all events are processed in (sim_time, priority, event_id) order, guaranteeing identical traces for the same seed.
- **Time precision:** all times rounded to 6 decimal places (μs resolution).

2.4 Evaluation Seeds and Averaging

Benchmark results are averaged over **30 seeds** (1–30) to smooth non-determinism introduced by Python’s hash randomization, which affects HEFT’s tie-breaking when multiple nodes have equal estimated finish times.

Win-regime unit tests (`test_win_regimes.py`) use fixed seeds (default seed=42) and **task pinning** (`pinned_to` field) to ensure completely deterministic, scheduler-independent results.

2.5 Workload Heterogeneity

The evaluation workloads are intentionally **heterogeneous** in both data sizes and compute costs. Uniform synthetic workloads tend to mask the advantages of flow-prioritization and criticality-aware schemes; heterogeneous workloads expose them.

2.5.1 Heterogeneous Data Sizes (2–30 MB)

Transfer sizes are drawn from a fixed cycling list:

[10, 2, 25, 5, 30, 8, 15, 3, 20, 6, 12, 28, 4, 18, 7, 22] MB

This gives a **15× range** (2 MB to 30 MB) with a roughly uniform spread on a log scale. The same list is cycled across all edges in a DAG, so every DAG instance has a mix of small, medium, and large transfers regardless of topology size.

Why it matters:

- **Makespan is dominated by the largest transfers.** A 30 MB flow on a congested link can cost 10× as much as a 3 MB flow on the same link. Schemes that route large flows first (**GB**) or onto uncongested paths (**GSD**) gain the most from heterogeneous sizes.
- **HEFT’s transfer-time estimates are size-scaled.** HEFT-2 uses bottleneck bandwidth b and estimates $\text{data_size}/b$ for each edge; large edges therefore pull heavily on the EFT calculation and influence task placement.
- **Small transfers are largely invisible to interference.** A 2 MB flow at 8.6 MB/s takes 0.23 s; a 30 MB flow takes 3.5 s. Deferral (**GSD-D**) provides more benefit when long transfers create sustained link contention.

2.5.2 Heterogeneous Compute Costs (150–1000 compute units)

Task compute costs are drawn from a fixed cycling list:

[500, 200, 800, 350, 1000, 150, 600, 450, 750, 300, 900, 250, 550, 700, 400, 850] units

This gives a **≈6.7× range** (150 to 1000 compute units). Node capacities are similarly heterogeneous:

[200, 100, 150, 80, 300, 120, 250, 180, 160, 90, 220, 140, 280, 110, 190, 170] cu/s

giving task runtimes from $150/300 = 0.5$ s to $1000/80 = 12.5$ s depending on node assignment.

Why it matters:

- **HEFT upward ranks spread widely.** A task with `compute_cost=1000` that feeds several high-cost successors has a much higher HEFT upward rank (`ranku`) than a leaf task with cost 150. This creates a clear ordering for **GC** (criticality-based routing) to exploit.
- **Task runtime variation staggers transfer start times.** On a 16-task grid, actual transfer start times may span several seconds, meaning the simultaneous-flow assumption implicit in the static greedy schemes is only partially correct. **GSD** benefits because it observes real congestion at the actual start time rather than a predicted window.
- **Placement is sensitive to compute cost vs. transfer cost.** HEFT-2 weighs the cost of sending a large data volume over a multi-hop path against the benefit of using an otherwise-idle fast node. With wide compute-cost variation, some tasks are worth the multi-hop transfer penalty; others are not. This is where HEFT-1 (which penalizes all multi-hop links) and HEFT-2 (which estimates them honestly) diverge most.

2.6 HEFT Scheduler Variants

HEFT (Heterogeneous Earliest Finish Time) decides *where* each task runs by estimating the Earliest Finish Time on every node and assigning each task to the node where it finishes soonest:

$$\text{EFT}(\text{task}, n) = \text{EST}(\text{task}, n) + \frac{\text{compute_cost}}{\text{node.compute_capacity}}$$

$$\text{EST}(\text{task}, n) = \max\left(t_n^{\text{free}}, \max_{\text{pred}} \left[\text{EFT}(\text{pred}, n_{\text{pred}}) + \frac{\text{data_size}}{\text{rate}(n_{\text{pred}}, n)} \right] \right)$$

The term $\text{rate}(n_i, n_j)$ is the pairwise transfer rate that HEFT uses for every (source node, destination node) pair. Because the SAGA library requires a fully-connected graph, a rate must be supplied for *every* pair, including pairs with no direct link. The two variants differ only in how they populate this matrix.

2.6.1 HEFT (Calibrated) — Runtime-Routing BW for All Pairs

Property	Value
Class	<code>SagaScheduler</code>
CLI / YAML	<code>scheduler: heft</code>
Adjacent pair	bandwidth from the runtime routing model
Same node	10,000 MB/s (effectively instant)
No direct link	bandwidth from the runtime routing model (0.001 MB/s if unreachable)

The original HEFT scheduler passes the *runtime* routing model directly into the pairwise rate matrix. When you run with `--routing shortest_path`, HEFT calls `ShortestPathRouting.get_path_bandwidth` for every (src, dst) pair; when you run with `--routing interference_aware_dynamic`, it delegates to `WidestPathRouting` (since dynamic models have no pre-simulation state) but at least uses the same code path.

Consequence: HEFT’s placement decisions are calibrated to the same routing scheme that will run at simulation time. If you choose S as the runtime routing scheme, HEFT sees S-based bandwidth estimates and places tasks at nodes that work well with S’s path choices. This alignment between planner and router is especially valuable when the runtime routing scheme has systematically different path preferences from widest-path.

When it equals HEFT-2: When the runtime routing scheme is `widest_path`, the original HEFT and HEFT-2 produce identical pairwise rate matrices and therefore identical placements. Differences emerge with other routing schemes (`shortest_path`, `shortest_hop`, etc.) on large topologies where path BW estimates diverge.

2.6.2 HEFT-1 — Penalty Rate for Non-Adjacent Pairs

Property	Value
Class	<code>Heft1Scheduler</code>
CLI / YAML	<code>scheduler: heft1</code>
Adjacent pair	actual link bandwidth
Same node	10,000 MB/s (effectively instant)
No direct link	0.001 MB/s (penalty)

Non-adjacent node pairs receive a near-zero rate (0.001 MB/s). HEFT computes a transfer time of `data_size / 0.001`, which is enormous — a 100 MB transfer is estimated at 100,000 seconds. HEFT therefore **almost never places communicating tasks on non-adjacent nodes**, concentrating placement on same-node or direct-neighbour assignments.

Consequence: the pairwise rate matrix is accurate for adjacent pairs and wildly wrong for all others. On a grid topology where the routing engine will happily route multi-hop (e.g. via `widest_path` or `GSD`), HEFT-1 makes placement decisions based on fictitious costs and may produce heavily unbalanced assignments that waste available network capacity.

2.6.3 HEFT-2 — Widest-Path Bottleneck Rate for All Pairs

Property	Value
Class	<code>Heft2Scheduler</code>
CLI / YAML	<code>scheduler: heft2</code>
Adjacent pair	widest-path bottleneck bandwidth
Same node	10,000 MB/s (effectively instant)
No direct link	widest-path bottleneck BW (0.001 MB/s if truly unreachable)

Every pair receives the bottleneck bandwidth of the best multi-hop path between those two nodes, computed via `WidestPathRouting` at scheduling time. HEFT sees accurate transfer time estimates for all reachable pairs regardless of whether a direct link exists.

Consequence: HEFT can legitimately spread tasks across the topology. A task assigned to a non-adjacent node incurs a real but correctly-estimated multi-hop transfer cost, so HEFT weighs that cost fairly against the compute benefit of using an otherwise-idle node.

Important caveat: HEFT-2 always uses `WidestPathRouting` internally for its BW estimates, regardless of the routing scheme used at simulation runtime (`SP`, `GSD`, etc.). If the runtime routing scheme chooses a different path — one with lower bottleneck bandwidth — the actual transfer time will exceed HEFT’s estimate. HEFT-2 estimates are unbiased on average but may diverge for specific flows.

2.6.4 Comparison

When to use each:

	HEFT (Calib.)	HEFT-1	HEFT-2
Rate source	runtime routing model	0.001 MB/s (penalty)	WidestPathRouting
Placement bias	matches runtime scheme	same-node / direct neighbours	any reachable node
Estimate accuracy	accurate for runtime scheme	accurate for adjacent only	accurate (widest-path)
Alignment with runtime	exact	independent	assumes widest-path BW
Risk	runtime model may delegate to widest-path anyway	wastes multi-hop capacity	overestimates BW if runtime routing picks narrower path
YAML key	heft	heft1	heft2

Table 1: HEFT scheduler variant pairwise rate matrix strategies.

- **HEFT (Calibrated)** is the best default for multi-hop routing schemes (S, SH, GS, GC, GB, GO). Its placement decisions account for the same bandwidth structure that the runtime router uses, minimising the gap between HEFT’s estimates and actual execution.
- **HEFT-1** is appropriate when the runtime routing is **direct** (single-hop only), or on `csma_bianchi` grids where co-location is intentionally preferred because multi-hop interference costs are prohibitive.
- **HEFT-2** is a convenience variant: it always uses widest-path BW regardless of runtime scheme. Equivalent to Calibrated HEFT when paired with **widest_path**; may diverge for other schemes.

3 Baseline Schemes

3.1 D — Direct Link

Property	Value
Class	<code>DirectLinkRouting</code>
Objective	Use direct (single-hop) link between source and destination
Algorithm	Lookup: <code>network.get_link_between(src, dst)</code>
Multi-hop	Not supported; returns <code>None</code> if no direct link exists
Interference awareness	<code>None</code>

The simplest scheme: if a direct link exists between two nodes, use it. If no direct link exists, the transfer fails. This is the Phase 2 baseline used to verify core simulation correctness on fully-connected topologies. Not suitable for sparse or grid networks where tasks are frequently placed on non-adjacent nodes.

3.2 W — Widest-Path

Property	Value
Class	<code>WidestPathRouting</code>
Objective	Maximize bottleneck (min-link) bandwidth
Algorithm	Modified Dijkstra with max-min semantics
Interference awareness	<code>None</code> —ignores concurrent traffic

Finds the path whose weakest link has the highest bandwidth. Good when link capacities vary widely (e.g., short vs. long hops have different PHY rates) but oblivious to runtime contention.

Also used by all other routing schemes as the **bandwidth oracle** during HEFT scheduling and as a fallback when no real-time state is available.

3.3 S — Shortest-Path (Minimum Transfer Delay)

Property	Value
Class	<code>ShortestPathRouting</code>
Objective	Minimize total transfer delay: $\min \sum_{\ell \in \text{path}} \frac{1}{b_\ell}$
Algorithm	Standard Dijkstra with edge weight $1/\text{bandwidth}$
Interference awareness	None

Edge weight: Each link ℓ contributes $1/b_\ell$ to the path cost, where b_ℓ is the link bandwidth in MB/s. The total path cost

$$\text{cost} = \sum_{\ell \in \text{path}} \frac{1}{b_\ell}$$

is proportional to $\text{data_size} \times \text{cost}$, i.e., the total transfer time for any fixed data size if the bottleneck were the average inverse-bandwidth. In practice, transfer time is determined by the *minimum* link bandwidth (bottleneck), but minimizing $\text{sum}(1/b)$ strongly disfavors slow links and tends to find high-throughput paths.

Comparison with previous definition: The prior definition minimized total latency ($\sum \text{lat}_\ell$), which on equal-latency grids reduced to minimum hop count. The new definition minimizes $\text{sum}(1/b)$, which is delay-aware: it prefers high-bandwidth paths even if they have more hops, and avoids slow bottleneck links regardless of their propagation delay.

Path	Hops	Old cost (sum lat)	New cost (sum $1/b$)
Direct, BW=30	1	0.001	0.0333
2-hop, BW=100 each	2	0.002	0.0200 ← chosen
2-hop, BW=50 each	2	0.002	0.0400
2-hop, BW=200 each	2	0.002	0.0100 ← chosen

Table 2: Old vs. new SP cost for equal-latency links. The new metric correctly prefers the high-bandwidth 2-hop path over a low-bandwidth direct link, and correctly orders high-BW paths above low-BW paths of equal length.

S concentrates all same-(src, dst) flows on the same pre-computed path, since routes are cached at load time. This causes contention when many flows share a path, which is the primary regime where interference-aware schemes outperform it.

3.4 SH — Shortest-Hop (Minimum Hop Count)

Property	Value
Class	<code>ShortestHopRouting</code>
Objective	Minimize hop count; tie-break by $\min \sum_{\ell \in \text{path}} 1/b_\ell$
Algorithm	BFS / modified Dijkstra with primary key = hops, secondary key = $\sum 1/b$
Interference awareness	None

Motivation: In `csma_bianchi` interference, each active link is an independent transmitter. A relay hop adds a second simultaneous transmission to the network, increasing the interference factor seen by all nearby flows. Minimizing hop count directly minimizes the number of new interfering links created per transfer, which motivated SH as a candidate improvement over S.

Cost function:

$$\text{key}(\text{path}) = (\text{hops}, \sum_{\ell} 1/b_{\ell})$$

Paths are sorted lexicographically: fewer hops always wins; among equal-hop paths, the lower $\sum 1/b$ wins (same tie-break as S).

When SH $\neq$ S: The two differ only when a 1-hop path through a low-bandwidth link (e.g. a diagonal at 56.6 m) has lower hop count than the cheapest $\sum 1/b$ path. S would pick the 2-hop high-bandwidth route (lower $\sum 1/b$); SH picks the 1-hop diagonal (fewer hops, higher bottleneck cost).

Empirical result (`csma_bianchi` 40 m grids): On the 4×4 and 7×7 evaluation grids, SH and S produce *identical* results for nearly all experiments. Where they differ (7×7 large DAG), S outperforms SH by 6% under HEFT-1. The reason: the grid’s high-BW cardinal links make the 2-hop cardinal route cheaper in $\sum 1/b$ than the 1-hop diagonal, so S’s 2-hop choice delivers higher bottleneck bandwidth despite the extra relay. The expected interference benefit of fewer hops is outweighed by the bandwidth penalty of the diagonal link. **Use S rather than SH on these grids.**

4 Static Greedy Schemes (GS, GC, GB, GO)

All four variants share the same two-stage architecture:

1. **HEFT** decides task-to-node placement and produces timing estimates.
2. **Greedy route planner** (`InterferenceAwareRouting.plan_routes()`) iterates over inter-node flows in a variant-specific order and, for each flow, picks the candidate path that maximizes *total system throughput* across all concurrently-active flows (including previously routed ones).

Routes are computed once at DAG injection and are fixed for the entire execution. The variants differ only in the order flows are considered.

4.1 Scoring Function (All Four Variants)

For each candidate path, the planner computes a global throughput score:

$$\text{score} = \sum_{\text{concurrent flows}} \text{effective_bottleneck_bandwidth}$$

where effective bandwidth for each flow accounts for:

- **Interference factor** from `csma_bianchi` (SINR degradation from hidden terminals + Bianchi MAC contention from conflict-graph neighbors)
- **Per-link fair sharing:** N flows on a link each get $1/N$ of the base (post-interference) bandwidth

The candidate path with the highest total-throughput score is selected.

4.2 Concurrency Detection

Because routes are planned before execution, the planner uses **HEFT timing estimates** to determine which flows are concurrent. Two flows are considered concurrent if their estimated time windows overlap:

$$[\text{start}_i, \text{end}_i] \cap [\text{start}_j, \text{end}_j] \neq \emptyset$$

This is an approximation; actual execution times differ due to queueing and bandwidth contention. GSD avoids this approximation by routing at actual transfer start time.

4.3 GS — Greedy by Start Time

Flow ordering	Ascending estimated transfer start time
Rationale	Route earlier flows first; later flows adapt around them
CLI	<code>--routing interference_aware --greedy-order start</code>

Earliest-first is the default and most natural ordering: it mirrors the temporal sequence in which flows will actually appear during execution.

4.4 GC — Greedy by Criticality

Flow ordering	Descending HEFT upward-rank of the <i>destination</i> task
Rationale	Critical-path flows get first pick of low-interference routes
CLI	<code>--routing interference_aware --greedy-order criticality</code>

Upward rank (**ranku**) measures how much downstream work depends on a task. Routing the most critical transfers first gives them the best paths, at the expense of less critical flows. Most effective when one or a few edges dominate the critical path and there exist alternative paths with significantly different BW.

4.5 GB — Greedy by Bytes

Flow ordering	Descending data size
Rationale	Large transfers dominate makespan; give them the best routes
CLI	<code>--routing interference_aware --greedy-order bytes</code>

When transfer sizes vary widely, the largest flows contribute most to makespan. GB prioritizes them. Less effective when all transfers are the same size (as in many synthetic benchmarks) or when multiple large transfers all target the same path.

4.6 GO — Greedy by Overlap

Flow ordering	Descending overlap degree (total temporal overlap with other flows)
Rationale	Flows competing with the most traffic should be routed first
CLI	<code>--routing interference_aware --greedy-order overlap</code>

Overlap degree is the sum of time-window overlaps between a flow and all other flows (using HEFT timing estimates). Flows competing for airtime with many others get priority. Most effective on dense grids with many parallel transfers.

5 Dynamic Greedy Scheme

5.1 GSD — Greedy by Start Time, Dynamic

Class	<code>DynamicInterferenceAwareRouting</code>
Objective	Maximize <i>this flow's</i> bottleneck bandwidth given actual state
When computed	At each <code>transfer_start</code> event
State used	Actual set of active links and per-link transfer counts
CLI	<code>--routing interference_aware_dynamic</code>

Unlike the static variants (which estimate concurrency from HEFT timing), GSD sees the *real* network state at the moment a transfer begins. The engine recomputes the route inside `handle_transfer_start`:

```
network_state = {
    "active_link_ids": self._get_active_link_ids(),    # links with >=1
    "link_transfer_counts": {lid: ls.num_transfers    # per-link count
                             for lid, ls in self.link_states.items()
                             if ls.num_transfers > 0},
}
new_path = routing_model.get_path(src, dst, network, network_state)
```

This lets GSD:

- React to transfers that finished early or late relative to HEFT estimates
- See sibling transfers that started moments earlier at the same sim-time (both TRANSFER_START events at $t = 5.0$ are processed sequentially; the second sees the first's active links)
- Avoid paths that are congested *right now*, not just predicted to be

Scoring: GSD scores each candidate path as:

$$\text{score}(\text{path}) = \min_{\ell \in \text{path}} \frac{b_{\ell} \cdot f_{\ell}}{\text{count}_{\ell} + 1}$$

where b_ℓ is the link’s nominal bandwidth, f_ℓ is the interference factor from currently-active links, and count_ℓ is the current transfer count on link ℓ . This is the per-flow bottleneck, not the total system throughput (unlike the static variants).

Trade-off: GSD optimizes for *this individual flow’s* bottleneck rather than total system throughput. It cannot coordinate across sibling flows the way the static planner can.

Delegation: During HEFT scheduling (before execution), GSD delegates to `WidestPathRouting` for bandwidth estimates, since no runtime state is available.

6 Deferral Scheme

6.1 GSD-D — Dynamic + Deferral

Class	<code>DeferralDynamicRouting</code>
Objective	Maximize flow bottleneck BW; defer if congestion is severe
When computed	At each <code>transfer_start</code> event
State used	Actual set of active links and per-link transfer counts
CLI	<code>--routing interference_aware_dynamic_deferral</code>

GSD-D extends GSD with a **deferral** mechanism. At each `transfer_start` event, the routing model computes the best path (identical to GSD) and then compares the effective bottleneck bandwidth to the no-contention baseline:

$$\text{ratio} = \frac{\text{effective_bottleneck_BW}}{\text{no_contention_BW}}$$

If $\text{ratio} < \text{deferral_threshold}$ (default 0.3), the engine **postpones** the transfer. Deferred transfers are re-evaluated after every `transfer_complete` event, when network conditions may have improved.

6.2 Deferral Mechanism

1. At `transfer_start`: routing model sets `should_defer` flag if effective BW is below threshold fraction of uncongested BW.
2. Engine checks the flag. If deferral is indicated *and* there are active transfers in the network (to ensure eventual retry), the transfer is stored in a deferred queue. The `transfer_start` trace event is suppressed.
3. At every `transfer_complete`: the engine re-evaluates all deferred transfers by rescheduling `transfer_start` events. Each re-evaluation computes a fresh path with updated state.
4. Starvation prevention: each transfer may be deferred at most `max_deferrals` times (default 3), after which it starts regardless of congestion.
5. Deadlock prevention: if no transfers are currently active, deferral is skipped (the transfer starts immediately).

6.3 Parameters

Parameter	Default	Effect
deferral_threshold	0.3	Defer if effective/no-contention BW ratio is below this
max_deferrals	3	Maximum times a single transfer can be deferred

6.4 When Deferral Helps

Deferral is most effective on **small, dense grids with large DAGs** (e.g., 4×4 with a 30-task pipeline), where:

- Many transfers compete for the same few links simultaneously
- Waiting 1–2 transfer completions frees enough airtime for a substantially faster path
- The DAG has enough parallelism that deferring one transfer does not stall the critical path

On sparse grids or small DAGs, contention is mild enough that deferral adds latency without benefit.

7 Configuration Reference

7.1 CLI

```
--scheduler {heft, heft1, heft2, cpop, round_robin, manual}
--routing {direct, widest_path, shortest_path, shortest_hop,
           interference_aware, interference_aware_dynamic,
           interference_aware_dynamic_deferral}
--greedy-order {start, criticality, bytes, overlap}
--routing-hop-cutoff N          # default 4
--deferral-threshold F          # default 0.3
--max-deferrals N               # default 3
--interference {none, proximity, csma_clique, csma_bianchi}
--interference-radius R         # for proximity model, default 15.0
```

7.2 YAML

```
config:
  scheduler: heft2          # heft1 or heft2
  routing: interference_aware
  greedy_order: start
  routing_hop_cutoff: 4
  interference: csma_bianchi
```


7.3 Shared Parameters

Parameter	Default	Effect
hop_cutoff	4	Maximum hops in candidate path enumeration
max_candidates	20	Maximum candidate paths evaluated per flow

8 Summary

Scheme	Route timing	Objective	Flow ordering	Interference input	Multi-hop
D	Load time	Direct link only	—	None	No
W	Load time	Max bottleneck BW	—	None	Yes
S	Load time	Min sum(1/bw)	—	None	Yes
SH	Load time	Min hops; tie-break sum(1/bw)	—	None	Yes
GS	DAG inject	Max system throughput	Est. start time	HEFT estimates	Yes
GC	DAG inject	Max system throughput	HEFT upward rank	HEFT estimates	Yes
GB	DAG inject	Max system throughput	Data size (desc)	HEFT estimates	Yes
GO	DAG inject	Max system throughput	Overlap degree	HEFT estimates	Yes
GSD	Transfer start	Max flow bottleneck BW	—	Actual active	Yes
GSD-D	Transfer start	Max flow bottleneck BW	—	Actual + deferral	Yes

Table 3: All routing schemes with key properties. “Route timing” is when paths are computed; “Interference input” is what state the scheme uses when choosing paths.

When to use each scheme:

- **D:** Only on fully-connected small test topologies.
- **W:** When link BW varies widely and contention is low. Good HEFT bandwidth oracle.
- **S:** Best on sparse grids / small DAGs / low CCR, where high-bandwidth paths also happen to be short. Outperforms SH on csma_bianchi grids because 2-hop high-BW paths beat 1-hop diagonal low-BW links both in bottleneck bandwidth and in practice.
- **SH:** Use only when you have strong prior evidence that relay interference (not bottleneck bandwidth) is the binding constraint. On standard 40 m grids this is not the case; SH and S tie or S wins.
- **GS/GC/GB/GO:** Useful when HEFT timing estimates are accurate and the DAG has enough structure to benefit from flow prioritization. GB is best with skewed transfer sizes; GC is best when one critical path dominates.
- **GSD:** Best when actual runtime congestion differs significantly from HEFT predictions (many concurrent sibling flows, variable task durations). Beats SP whenever parallel alternative paths exist.
- **GSD-D:** Best on small dense grids with large DAGs where many flows compete for the same few links simultaneously. Wins the 4×4 /large-DAG benchmark (-56% vs. W, -32% vs. S).